# Diccionario de Palabras Clave — Semana 3

BOOTCAMP: DE CERO A DATA ENGINEER | DATABRICKS SQL | CONSTRUIR | CLASE GRUPAL

Términos técnicos de la Semana 3: arquitectura Medallion, Delta Lake, Git, Spark y gobernanza. Referencia rápida para ejercicios y entrevistas — sin conocimientos previos requeridos.

| | |
|---|---|
| **Git y Repos**<br>Pull, Push, Commit, Branch, PAT | **Workspaces**<br>Unity Catalog, permisos, ambientes |
| **Medallion**<br>Landing → Bronze → Silver → Gold | **Delta Lake**<br>Parquet, ACID, Time Travel, DML |
| **Spark & SQL**<br>Cluster, Driver, Executor, Vistas | **Gobernanza**<br>Incremental, MERGE, EDA, Mesh |

SEMANA 3

# Git y Repositorios en Databricks

## Operaciones principales

| | |
|---|---|
| `pull` | Traer cambios del remoto a Databricks |
| `push` | Subir commits al remoto |
| `commit` | Registrar cambios con mensaje |
| `merge` | Integrar una rama en otra |

## Conceptos de flujo

### Branch (rama)

Línea de trabajo paralela; se integra vía PR

### Pull Request (PR)

Revisión antes de fusionar en `main`

### PAT (Personal Access token)

Token de  Databricks

### Historial de commits

Quién, cuándo y qué se modificó — linaje del código

⚠️ Si el `push` falla con error de permisos → revisar escritura en el PAT o en el repo privado.

SEMANA 3

# Workspaces y Unity Catalog



## Workspace (ambiente)

Entorno Dev / Stage / Prod con permisos y compute propios



## Workspace (carpetas)

Árbol de notebooks en la barra lateral — distinto al ambiente



## Unity Catalog

Gobernanza central: catálogos, esquemas, tablas y permisos



## Catálogos DEV / STAGE / PROD

Separación por ciclo de vida — evita borrar producción por error

## Metadatos

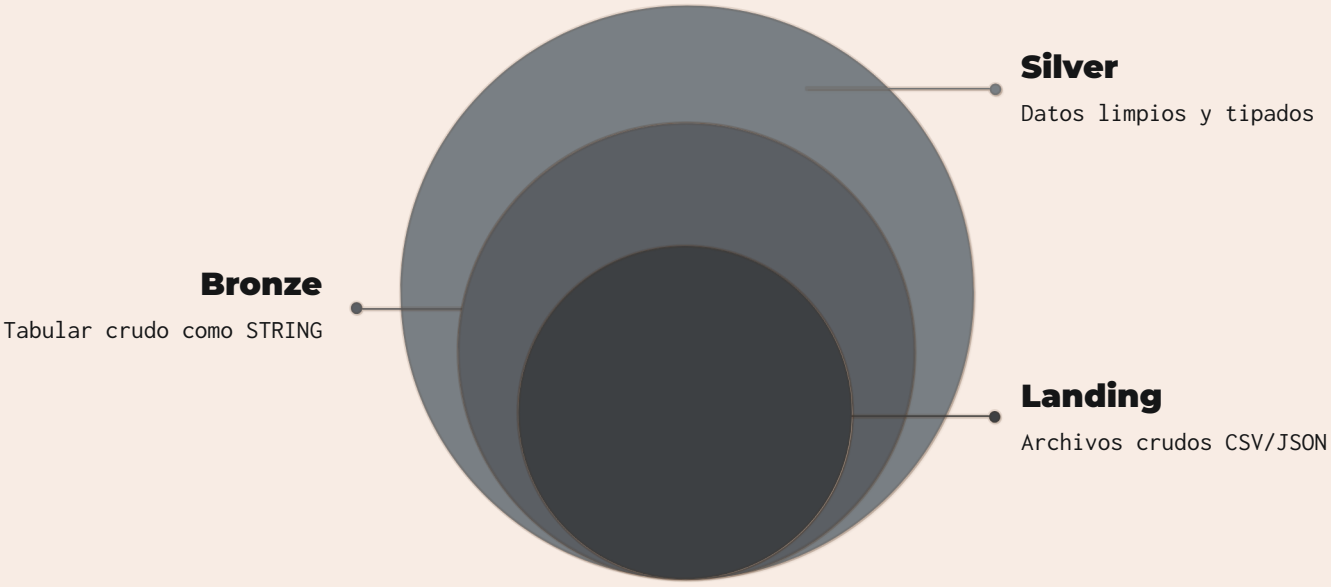
Información sobre ubicación, esquema, permisos y versiones (Delta log, catálogo)

## Permisos granulares

Acceso a nivel catálogo → esquema → tabla → vista → columna según el rol

SEMANA 3

# Capas Medallion y Arquitectura Analítica




## OLTP

Transaccional: compras, pagos — respuesta rápida, sin reportes pesados

## Data Warehouse

Analítico relacional, esquema fijo — limitado frente a variedad cruda

## OLAP

Analítico: agregaciones y estrategia — separado para no tumbar OLTP

## Data Lake

Muchos formatos y volúmenes — sin gobernanza se vuelve pantano

## Linaje

Rastrear origen y transformaciones — más fácil con capas claras

## Data Swamp

Lake ingobernable: muchos archivos, poca confianza ni estándares

## Archive

Datos viejos fuera del flujo diario — conservados por legalidad

## Lakehouse

Unifica warehouse + lake — Delta habilita transacciones sobre archivos

SEMANA 3

# Delta Lake y Formatos de Archivo

## Stack de formatos

01

### Parquet

Columnar y comprimido — base analítica

02

### Delta

Parquet + log transaccional — default en Databricks

03

### ACID

Transacciones atómicas — evita estados rotos

04

### Time Travel

Leer versiones anteriores — útil ante errores o borrados

## Operaciones DML

| | |
|---|---|
| INSERT | Agregar nuevas filas |
| UPDATE | Modificar filas existentes |
| DELETE | Eliminar filas |
| MERGE | Upsert: insertar o actualizar según clave |

## Tipos de datos

### Estructurados

Filas, columnas y tipos claros

### Semiestructurados

JSON, XML — estructura anidada a explotar

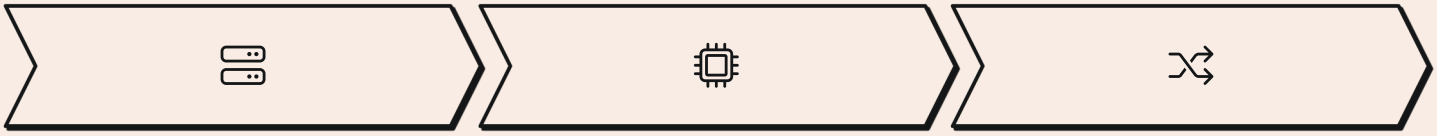
### No estructurados

Imágenes, PDF, video — requieren ML/OCR

Schema evolution: permite nuevas columnas con reglas — tiene costos si no se controla.

SEMANA 3

# Cluster Spark, Vistas y SQL



### Driver

Coordina el job y reparte trabajo

### Executors

Ejecutan tareas sobre particiones

### Shuffle

Redistribuye datos entre nodos — operación cara

## Vistas y tablas

| | |
|---|---|
| **Vista** | Query guardada — re-ejecuta lógica en cada consulta |
| **Tabla persistida** | Datos materializados en storage Delta |
| **Vista materializada** | Precomputada con política de refresco |
| **Trade-off** | Duplicar datos vs. pagar recomputo de vistas |

## Filtrado SQL

```
-- LIKE: comodines simples
WHERE campo LIKE '%pilar'    -- termina con
WHERE campo LIKE '%pilar%'   -- contiene

-- RLIKE: expresión regular (Java)
WHERE campo RLIKE '.*pilar$'
```

## Modelado

### Dimensional

Hechos + dimensiones — se profundiza en Gold

### Data Vault 2.0

Hubs, links y satélites — vive en Silver

SEMANA 3

# Cargas, Gobernanza y Calidad

## Estrategias de carga

| | |
|---|---|
| **Incremental** | Solo registros nuevos o cambiados |
| **Watermark** | Marca temporal para saber desde dónde traer el delta |
| `MERGE` | Insertar o actualizar según clave (upsert) |
| **Hash de fila** | Huella para detectar cambios o duplicados |
| `TRUNCATE` | Vaciar tabla — mala práctica habitual en prod |
| **Reproceso** | Re-ejecutar pipeline sobre histórico ante cambios |

⚠️ `TRUNCATE` como estrategia habitual en prod es mala práctica — preferir incremental + `MERGE`.

## Gobernanza y calidad

### EDA

Exploración en Bronze: nulos, cardinalidad, outliers antes de Silver

### Data Mesh

Cada dominio es dueño de sus datos como producto

### Data Product

Dataset con owner, consumidores y SLA — no un archivo suelto

### Auditoría

Demostrar qué datos había y qué les pasó — Bronze y Landing ayudan

### Compliance

Cumplir GDPR, CCPA y normas de retención de datos personales

Instructor: Luciano Argolo | www.LucianoArgolo.com